
Multi Agent Cooperation in the StagHunt environment

Marco Crisafulli

Abstract

The StagHunt environment available here contains 3 multi-agent grid-based stochastic games, the aim of this project is to explore prosocial behavior in multi-agent reinforcement learning. In game theory, the stag hunt describes a situation or game where participants would be better off cooperating to achieve a more ambitious goal but can choose a safer option that protects them from a bad outcome. This sets up a stylized conflict between safety and social cooperation based on the rewards of each option. Training agents that cooperate in such environments can be a tricky task; I propose a solution to achieve cooperation and the results of my experiment.

1 Introduction

MARL algorithms can be categorized based on the information available during the training and execution of policies. During training, a MARL algorithm can be restricted to only use local information observed by each agent ("de-centralized training") or it can use information about all agents in the multi-agent system ("centralized training"). The execution can be categorized in a similar way: each agents' policy is conditioned on the local information of the agent ("Decentralized execution") or each policy is conditioned on the availability of information from all agent ("Centralized execution"). Centralized training and decentralized execution (CTDE) represents the third paradigm of MARL and it is the one I chose to use because it combines the advantages of a centralized training with the benefit of decentralized execution. I will discuss the game and challenges followed by the actual implementation.

2 Understanding The Environment

The StagHunt environment provided is based on the OpenAI gymnasium library. Three games are available and the current implementation of the algorithm is meant to be used in the Hunt game: Two agents start off on the top two corners of a grid like world, they can move up, down, left, right or stay in position. Two plants are spawned in random positions and a stag wanders in this world. If an agent ends up on top of a plant, it forages it and gets a small reward, if an agent is caught alone by the stag, it gets mauled and punished with a bad reward. If two agents are on top of a stag together, they earn the stag reward which is bigger than the forage reward. The game can be therefore visualized as a social dilemma matrix:

Table 1: Stag Hunt as a normal-form game, standard configuration of the game provided.

		Agent 2	
		Hunt	Forage
Agent 1	Hunt	5,5	-5,1
	Forage	1,-5	1,1

2.1 Observations and first design choice

The environment can be customized to serve observations in different forms: tuple of coordinates or rgb arrays. I have chosen to use the coordinates as they represent the information in a compact form. First design choice: I wrapped the environment using the methods provided by gymnasium and I have extended the standard observation provided with 2 added values, one representing the distance between agents (This value is the same for each agent) and another one representing the distance between the agent and the stag (This depends on the position of the individual agent). The distance type is the euclidean distance because it is sufficiently expressive for the job. This added features aim to "specialize" the network during training to recognize the link between stag coming after you and you being close to the the other player.

Table 2: Observation space augmentation from (2,10) to (2,12).

Agent	Original (10 dims)	Augmented (+2 dims)
Agent 0	$[x_0, y_0, x_1, y_1, s_x, s_y, xp_0, yp_0, xp_1, yp_1]$	$+ [d_{aa}, d_{a_0s}]$
Agent 1	$[x_1, y_1, x_0, y_0, s_x, s_y, xp_0, yp_0, xp_1, yp_1]$	$+ [d_{aa}, d_{a_1s}]$

2.2 Rewards

Thanks to the trick of the augmented observations I managed to achieve cooperation without changing the rewards. One attemp was made by adding reward shaping to make cooperation emerge faster but as I will show in the results it is not needed. The reward table is presented as Table 1.

3 Behavioural considerations

3.1 Non-stationarity

In multi-agent environments, the behavior of an agent is not only determined by its own actions but also influenced by the actions of other agents in the environment. Therefore an agent has to learn a correct policy around the policy of other agents, but in MARL, the policies of other agents are changing as they learn. This problem can be formalized as the Non-stationarity in MARL which means that from the perspective of any single agent, the environment appears to be changing over time not because the physical rules changed, but because the other agents are also learning and changing their behavior.

3.2 homogeneous agents

Let's introduce the concept of homogeneous agents: A game in which the agents are homogeneous intuitively means that all agents share the same or similar capabilities and characteristics, such as the same set of actions, observations, and rewards. An environment has:

1. **Weak homogeneity** if for any joint policy $\pi = (\pi_1, \dots, \pi_n)$ and permutation $\sigma : \mathcal{I} \rightarrow \mathcal{I}$,
$$U_i(\pi) = U_{\sigma(i)}(\pi_{\sigma(1)}, \dots, \pi_{\sigma(n)}), \quad \forall i \in \mathcal{I}.$$
2. **Strong homogeneity** if it is weakly homogeneous and there exists an optimal joint policy $\pi^* = (\pi_1^*, \dots, \pi_n^*)$ with $\pi_1^* = \pi_2^* = \dots = \pi_n^*$.

The Stag Hunt environment satisfies both weak and strong homogeneity.

3.3 Shared Parameters vs Shared Experience

One way to leverage this property is by using shared parameters when updating each agent's neural network, but a drawback is that strong homogeneity is a strong assumption and that might be difficult to verify. If agents share parameters, the policies of the agents are identical. Nonetheless, parameter sharing is computationally efficient because it requires learning just one set of policy (or value, or both) network parameters. An alternative method is to maintain separate parameters for each agent while still sharing the trajectories (experiences) collected by all agents. Thus, under the assumption of

weak homogeneity, an agent could sample and learn from a transition collected by a different agent. In contrast to parameter sharing, experience sharing does not require that the optimal joint policy $\pi^* = (\pi_1, \pi_2, \dots, \pi_n)$ consists of identical individual policies ($\pi_1 = \pi_2 = \dots = \pi_n$). Another advantage is that any successful policy (that achieves higher return in the long term) by an agent could be used by other agents to learn as it is stored inside the shared replay buffer. Furthermore, a shared replay buffer will contain more (and more recent) experiences than an individual buffer.

3.4 Second Design Choice

For the reasons discussed above, I have chosen to implement a Shared Replay buffer, and a natural candidate that uses it is the Independent Deep Q-Learning algorithm (IDQN with shared replay buffer).

4 The Agents

As a consequence of the two design choices presented above:

- **Augmented Observations:** implemented with a gym.wrapper
- **Experience sharing:** implemented with IDQN with shared replay buffer

Each agent must be equipped with a "brain" (called agentNetwork in the context of this project). Since I decided to use IDQN, my implementation consists of two networks per agent: a value network and a target network.

4.1 Value Network

The Value Network receives a state as its input and outputs its estimated action value for each possible action. The StagHunt environment accepts discrete action spaces and that is a requisite for any DQN implementations. The net is a feedforward neural network with an observation as input, 2 layers of 96 neurons with ReLU activation function and a vector of action values as output. The input observation consists of a vector of 12 float32 numbers shown in Table 2, the output vector consists of the q-values of the 5 possible actions. All agents share a common replay buffer $\mathcal{D}$, which stores transitions generated during training. At each update step, a mini-batch is sampled uniformly from $\mathcal{D}$ and is used to compute the TD loss for each agent independently. For agent i , the loss is:

$$\mathcal{L}_i(\theta_i) = \mathbb{E}_{(s,a,r,s',d) \sim \mathcal{D}} \left[\left(r + \gamma(1-d) \max_{a'} Q_i(s', a'; \theta_i^-) - Q_i(s, a; \theta_i) \right)^2 \right],$$

where θ_i and θ_i^- are the parameters of the value network and the target network of agent i respectively and $d \in \{0, 1\}$ indicates whether the next state s' is terminal ($d = 1$) or not ($d = 0$).

4.2 Target Network

The moving target problem in DQN arises when the same network is used to compute both the Q-value prediction and the TD target, causing the target to shift with every parameter update. For this reason we introduce the Target Network that will be dimensioned as the value network. **Hard target update:** Every N training steps, we copy the parameters of the value network θ to the target network θ^- :

$$\theta^- \leftarrow \theta.$$

N is a hyperparameter controlling how often the frozen target network is refreshed

4.3 Optimizer

To update the weights of the value net the optimizer chosen is the Adam Optimizer. Adam adapts the learning rate per parameter by combining momentum with adaptive gradient scaling, which typically accelerates convergence in deep RL settings

4.4 Action Selection

The action selection for IDQN follows an ϵ -greedy policy. With probability ϵ , the agent selects a uniform random action to encourage exploration; with probability $1 - \epsilon$, it selects the greedy action $a = \arg \max_a Q_i(s, a; \theta_i)$ to exploit its current estimate of the value function.

4.5 Save and Load

The save and load methods allow efficient saving and loading of the weights of the two networks and the optimizer after training. They are mainly used to test different iterations without losing the weights.

5 Training Loop and Shared Buffer

I decided to keep the training loop as simple as possible while adhering to the algorithm of choice. TensorBoard is included to monitor losses via a web interface.

5.1 Shared Buffer

The Shared Buffer doesn't store the agent id since all transitions are stored identically.

$\mathcal{D} \leftarrow \emptyset$ of capacity C
add($o_t, a_t, r_t, o_{t+1}, d$): store at ptr , then $ptr \leftarrow (ptr + 1) \bmod C$
sample(B): return B uniform random entries from $\mathcal{D}$

5.2 train_one_epoch

Handles an episode of max_timestep duration which can be configured. Each episode consists of:

Algorithm 1 train_one_epoch

```
1: for each agent  $i$  do
2:   Select action ( $a_t^i$ ) according to  $\epsilon$ -greedy policy
3: end for
4: Apply actions ( $a_t^1, \dots, a_t^n$ ); collect rewards  $r_t^1, \dots, r_t^n$  and next observations  $o_{t+1}^1, \dots, o_{t+1}^n$ 
5: for each agent  $i \in \{1, \dots, n\}$  do
6:   Store transition ( $o_t^i, a_t^i, r_t^i, o_{t+1}^i, d$ ) in shared replay buffer  $\mathcal{D}_{\text{shared}}$ 
7: end for
8: When  $|\mathcal{D}| \geq B$ , sample  $\mathcal{B} \sim \mathcal{D}$ , update  $\theta_i$  via  $\nabla \mathcal{L}_i(\theta_i)$ , and every  $N$  steps set  $\theta_i^- \leftarrow \theta_i$ .
9: return ( $o_{t+1}, d, \{\ell_i\}, \{r_i\}$ )
```

5.3 train

To further encourage exploration I have modelled *epsilon* parameter as a variable quantity, with a dedicated start, end and decay, fully configurable in the config.

Algorithm 2 Training Loop

```
1:  $\epsilon \leftarrow \epsilon_{\text{start}}, t \leftarrow 0$ 
2: for each episode do
3:    $o \leftarrow \text{env.reset}()$ 
4:   for each step until done do
5:     ( $o_{t+1}, d, \ell, r$ )  $\leftarrow \text{train\_one\_epoch}(t, o, \epsilon)$ 
6:      $t \leftarrow t + 1, \epsilon \leftarrow \max(\epsilon_{\text{end}}, \epsilon \cdot \epsilon_{\text{decay}}), o \leftarrow o_{t+1}$ 
7:     Record losses  $\ell$  and return  $r$ 
8:   end for
9: end for
```

5.4 Metrics

Throughout the code there are many lines dedicated to pure metrics collection which are printed and inform us at training time about the progress of the training. These metrics are mainly used to correct trajectory and validate the various iterations of the algorithm.

6 Main and configuration

To keep everything organized and readable, I decided to decouple the main entry point for the training loop and the configuration parameters

6.1 Config

A configuration instance ("Config") is created from a dataclass that contains all the parameters applied to control the environment, all the hyperparameters used in the networks and the dimension of the hidden layers of the networks. Thanks to a tidy and organized config file I am able to change everything I need on the go, allowing me to partake in tedious tasks like hyperparameter search and tuning.

6.2 Main

The Main section handles the wrapper for the augmented observations, applies the configuration parameters to the environment, instantiates the agents, the shared buffer and the trainer.

7 Results

7.1 Defining Cooperation

We consider environments with exactly two pure Nash equilibria: one efficient (high welfare) and one inefficient (low welfare). We define *cooperation* as the action profile that leads to the efficient equilibrium when chosen by both agents. For example, in the Stag Hunt (Table 1), the cooperative action is Hunt, while the safe, non-cooperative action is Forage.

7.2 Methods and results

I tried training the two agents with various epochs and with both reward shaping and no reward shaping. The longest training run I launched was 1500 epochs - 500 timesteps for each episode for a total of 750000 timesteps, while the "Standard" run to tune each parameter is 500 epochs - 500 timesteps each epoch for a total of 250000 timesteps. Depending on the seed we can observe more "variance" during training, but cooperation is achieved in all the runs. The hyperparameter configuration is accessible in the repo.

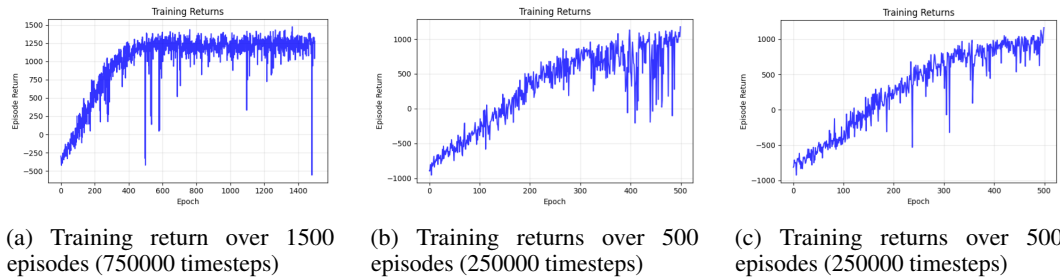


Figure 1: Training Results with no reward shaping, different seeds, episode return is cumulative, epochs on X axis

7.3 Cooperation

Each epoch has a fixed duration of 500 timesteps, the theorethical upper limit of the reward is then 5000 cumulative, assuming cooperation in each single step and assuming the stag in on top of the agents all the time. A more realistic expectation is a joint cooperative action every 5 timesteps (the time it takes for the stag to run away after getting caught and move back to the agents) which brings our expected reward at around 500 per agent in case of cooperation. As we notice from the plot (a), longer training times allow agents to even exceed this expectation by a small margin (with peaks around 1200 cumulative or 600 per agent). Cooperation emerges around 200000 timesteps.

7.4 Cooperation Rate

In order to present the results of the experiment on repeated episodic games after training is done, I have extracted a metric that I call the Cooperation Rate. It measures the amount of times the reward given by the environment to the agents is the joint greedy action (5,5) with respect to the total rewards. Another metric is the Cooperation occurrence that measures whether cooperation occurred in a given episode. The cooperation rate consistently sits around 20%. Cooperation occurs in >80% of episodes.

7.5 Cooperation visualized

The emergent cooperative behavior exhibits a clear spatial pattern. The stag follows the nearest agent; therefore, the optimal policy (the "META" Most Effective Tactic Available) is for both agents to occupy the same cell and remain stationary (occasional drift does not affect cooperation equilibrium). The learned cooperation manifests itself as a simple but effective wait-and-hunt strategy that exploits the environment's rule that the stag tracks the closest agent.

7.6 Results

The observed high standard deviations can be reduced via reward shaping that reinforces cooperation; cooperation occurred at least once in 92 of the 100 evaluation episodes.

Table 3: Per-Agent Mean Return, Confidence Interval, Hunt, Forage and Maul Reward frequency per-episode, averaged over 100 episodes. (Seed=456456) ($CI_{95} = \bar{x} \pm 1.96 \cdot s / \sqrt{n}$)

Agent	Mean return	95% CI	Hunt Frequency	Forage Frequency	Maul Frequency
0	443.15	[394.29, 492.01]	100.69	0.75	-12.21
1	426.32	[373.03, 479.61]	100.69	0.71	-15.57

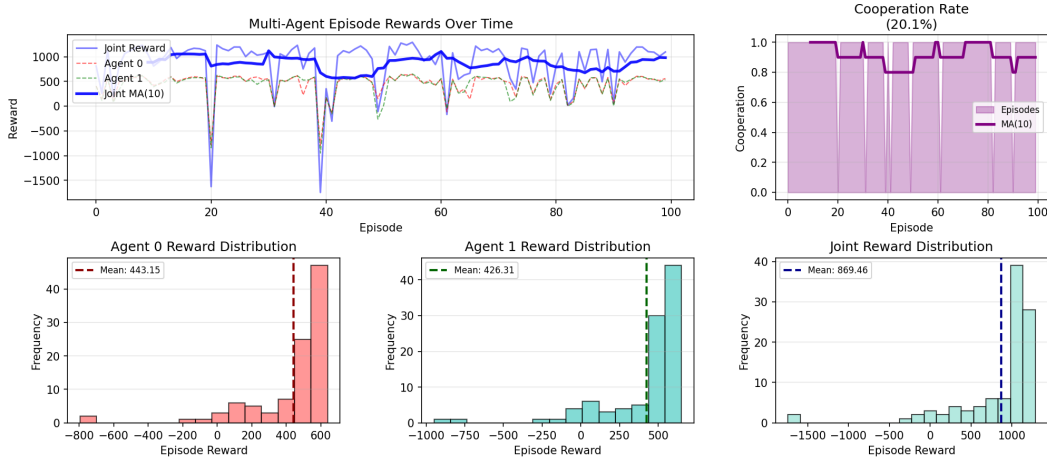


Figure 2: Evaluation of strongest pair of agents (750000 training timesteps) across 100 episodes, Agent 0 (443.15 ± 249.3) and Agent 1 (426.32 ± 271.9). Cooperation rate of 20.1%, Graph shows Cooperation Occurrence 7.4 (either 1 or 0) and Moving Average(10 episodes)